PRADITA
University

IF140303-Web Application Development

## Session-04:

# Atom, Map, Tuple, List, Keyword List

PRU/SPMI/FR-BM-18/0222

**Alfa Yohannis**



# Contents

PRADITA University


1 Atom
2 Map
3 Tuple
4 List
5 Keyword List
6 Converting Between Data Structures
7 Exercises
8 Summary

# Atom: The Basics

PRADITA University

In Elixir, an **atom** is a constant whose value is its own name. Atoms are frequently used to label values or represent specific concepts within a program. They are efficient and lightweight, making them ideal for pattern matching and as labels.
Atoms are created by prefixing a name with a colon (`:`).

```
:ok        # An atom with the value :ok
:error     # An atom with the value :error
:my_value # An atom can also use snake_case
```

# Using Atoms in Elixir

PRADITA University

Atoms are commonly used for control flow in **pattern matching**, as keys in **keyword lists**, and to represent **module names**. Since atoms are immutable, they provide a reliable way to reference named values.

```elixir
status = :ok

case status do
  :ok -> "Operation successful"
  :error -> "Operation failed"
end
#=> "Operation successful"

# Atoms as keys
options = [mode: :admin, visible: true]
```

# Map: The Basics

PRADITA University

In Elixir, a **Map** is a key-value data structure where keys can be of any type. Maps are unordered, meaning the key-value pairs are not stored in any specific sequence. They are created using the %{} syntax, and values can be accessed using their key.

```elixir
# Creating a map
user = %{"name" => "Alice", "age" => 30}

# Accessing a value
user["name"]  #=> "Alice"
```

# Modifying a Map

PRADITA University

To add or update an entry in a Map, you use the `Map.put/3` function. To remove an entry, you use the `Map.delete/2` function. Since Elixir data is immutable, these functions return a **new map** rather than changing the original one.

```elixir
user = %{"name" => "Alice", "age" => 30}

# Add a new key-value pair
updated_user = Map.put(user, "city", "New York")
#=> %{"name" => "Alice", "age" => 30, "city" => "New York"}

# Remove a key-value pair
final_user = Map.delete(user, "age")
#=> %{"name" => "Alice"}
```

# Tuple: The Basics

PRADITA University

A **Tuple** is an ordered collection of elements. A key characteristic of tuples is that they are **immutable**, meaning once a tuple is created, it cannot be changed. They are commonly used for returning multiple values from a function.
Tuples are created with curly braces `{}`, and elements are accessed by their zero-based index using the `elem/2` function.

```
# Creating a tuple
person = {"Alice", 30, "New York"}

# Accessing an element by index
elem(person, 0)  #=> "Alice"
```

# Working with Immutable Tuples

PRADITA University

Since tuples are immutable, you cannot directly add, update, or remove elements. Any "modification" operation will create and return a **new tuple**, e.g., the `Tuple.append/2` function doesn't change the original tuple; it returns a new one with the element added to the end. The same principle applies to removing elements.

```
person = {"Alice", 30}

# "Appending" creates a new tuple
new_person = Tuple.append(person, "Engineer")
#=> {"Alice", 30, "Engineer"}

person #=> {"Alice", 30} (The original is unchanged)
```

# Lists: Creation and Adding Elements

PRADITA University

A **List** in Elixir is an ordered collection of elements. Lists are ideal for variable-length data. All Elixir data is **immutable**, so "adding" an element to a list actually creates and returns a **new list**. The most efficient way to add an element is to the front (or head) of the list using the cons operator `[ | ]`. You can also insert an element at any index.

```elixir
# 1. Create a list with square brackets
list = [1, 2, 3]

# 2. Add an element to the head (very fast)
new_list = [0 | list]
#=> [0, 1, 2, 3]

# 3. Insert an element at a specific index
inserted_list = List.insert_at(list, 1, 99)
#=> [1, 99, 2, 3]
```

# Lists: Concatenating and Removing Elements

To combine two lists, you can use the concatenation operator `++/2`. This is the most common and idiomatic way to join lists in Elixir. To remove an element, the `List.delete/2` function is used. It scans the list and removes the **first occurrence** of the specified element. Both of these operations are immutable and return a **new list**.

```elixir
# 1. Concatenate two lists using the ++ operator
list1 = [1, 2]
list2 = [3, 4]
new_list = list1 ++ list2
#=> [1, 2, 3, 4]

# 2. Delete the first occurrence of an element
list_with_duplicates = [10, 20, 30, 20]
updated_list = List.delete(list_with_duplicates, 20)
#=> [10, 30, 20]
```

# Lists: Accessing the Head Element

PRADITA University

Accessing elements in an Elixir list is most commonly done by interacting with its **head** (the first element) and its **tail** (the rest of the list). The most idiomatic way to do this is with **pattern matching**, which lets you deconstruct the list into variables. For convenience, Elixir also provides the `hd/1` function as a direct shortcut to get the head.

```
list = [:a, :b, :c, :d]

# 1. Using pattern matching to get the head
[head | _tail] = list
# head is now :a

# 2. Using the hd/1 function for a direct shortcut
hd(list)
#=> :a
```

# Lists: Accessing the Tail and by Index

RADITA niversity

To get all elements of a list **except for the first one**, you can use the `tl/1` function. This is the counterpart to the `hd/1` function and returns the "tail" of the list. To access an element by its specific numerical position, you should use `Enum.at/2`. Note that this can be a slow operation on very long lists, as Elixir must traverse the list from the beginning to find the element.

```elixir
list = [:a, :b, :c, :d]

# 1. Get the tail of the list (all but the head)
tl(list)
#=> [:b, :c, :d]

# 2. Get the element at a specific index (zero-based)
Enum.at(list, 2)
#=> :c
```

# Advanced: Index Access with Recursion

This example implements index-based access from scratch using a recursive function with pattern matching. It works by repeatedly traversing the list's tail and decrementing the index, returning the current head when the index reaches zero.

```elixir
defmodule ListUtils do
  # Base case: when index is 0, return the head
  def get_element_at([head | _tail], 0), do: head
  # Recursive step: call again on the tail with index - 1
  def get_element_at([_head | tail], index) when index > 0 do
    get_element_at(tail, index - 1)
  end
  # Edge case: return nil for an empty list
  def get_element_at([], _index), do: nil
end
list = [1, 2, 3, 4]
IO.puts(ListUtils.get_element_at(list, 2)) # Outputs: 3
```

# Lists: Finding an Element's Index by Value

To find the position of an element in a list based on its value, you can use the `Enum.find_index/2` function. This function iterates through the list and returns the **zero-based index** of the **first element** for which the provided anonymous function returns a truthy value. If no element matches the condition, it returns `nil`.

```
list = [10, 20, 30, 40, 30]

# Find the index of the first element that equals 30
index = Enum.find_index(list, fn x -> x == 30 end)

IO.puts(index) # Outputs: 2
```

# Advanced: Finding an Index with Recursion

This custom function finds an element's index using **recursion** and **pattern matching**. It defines separate clauses to handle the three key scenarios: the base case when a value is found, the recursive step to traverse the rest of the list, and the failure case for an empty list.

```elixir
defmodule ListUtils do
  # Case 1: The list is empty, so the value was not found.
  def index_of([], _value, _index), do: nil
  # Case 2: The head of the list matches the value. Return the index.
  def index_of([value | _tail], value, index), do: index
  # Case 3: No match. Recurse on the tail with an incremented index.
  def index_of([_head | tail], value, index) do
    index_of(tail, value, index + 1)
  end
end
list = [10, 20, 30, 40]
index = ListUtils.index_of(list, 30, 0) #=> 2
```

# Keyword Lists: The Basics

PRADITA University

A **Keyword List** is a special type of list in Elixir that's commonly used for passing optional values to functions. Under the hood, it's a list of two-element tuples where the first element of each tuple is an **atom**. Elixir provides a convenient syntax for creating and accessing them.

```elixir
# Convenient syntax for creation
options = [name: "Alice", age: 30]

# The underlying structure is a list of tuples
# [{:name, "Alice"}, {:age, 30}]

# Accessing a value is simple
options[:name] #=> "Alice"
```

# Modifying a Keyword List

PRADITA University

Since Keyword Lists are just lists, you can add a new key-value pair by prepending it. If a key already exists, this effectively "updates" it, as functions typically read the first value they find. To explicitly remove an element, the `Keyword` module provides helper functions like `Keyword.delete/2`.

```elixir
options = [name: "Alice", age: 30]

# Add :city, creating a new list
new_options = [city: "New York" | options]
#=> [city: "New York", name: "Alice", age: 30]

# Remove a key, creating another new list
final_options = Keyword.delete(new_options, :age)
#=> [city: "New York", name: "Alice"]
```

# Converting Between Tuples and Lists



In Elixir, it's often necessary to convert between different data structures. The standard library provides simple, built-in functions to handle these transformations, most commonly between **tuples** and **lists**.

```elixir
# 1. Convert a Tuple to a List
my_tuple = {:ok, "some data", 123}
my_list = Tuple.to_list(my_tuple)
#=> [:ok, "some data", 123]

# 2. Convert a List to a Tuple
another_list = [:error, "not found"]
another_tuple = List.to_tuple(another_list)
#=> {:error, "not found"}
```

# Converting Between Maps and Keyword Lists

Elixir provides simple, built-in functions to convert between the general-purpose **Map** and the option-passing **Keyword List**. This is a common operation when dealing with function arguments or configuration data.

```elixir
# 1. Convert a Keyword List to a Map
# Use Enum.into/2 with an empty map as the target.
keyword_list = [name: "Budi", age: 25]
map = Enum.into(keyword_list, %{})
#=> %{name: "Budi", age: 25}


# 2. Convert a Map to a Keyword List
# The Map.to_list/1 function handles this directly.
map = %{name: "Budi", age: 25}
keyword_list = Map.to_list(map)
#=> [name: "Budi", age: 25]
```

# Converting a List of Tuples to a Keyword List

To convert a regular list into a **Keyword List**, the source list must follow a specific format: it must be a **list of two-element tuples**, where the first element of each tuple is an **atom**.
While Elixir often treats both structures identically, you can ensure the special keyword syntax is used with the `Enum.into/2` function.

```elixir
# The source must be a list of 2-element tuples with atom keys.
list_of_tuples = [{:name, "Budi"}, {:age, 25}]

# Enum.into/2 converts it to the keyword list syntax.
keyword_list = Enum.into(list_of_tuples, [])

# The underlying data is the same, but the representation is new.
IO.inspect(keyword_list)
#=> [name: "Budi", age: 25]
```

# Converting Between Tuples and Keyword Lists

While there isn't a single function to directly convert between a **Tuple** and a **Keyword List**, you can easily compose functions to achieve the transformation. The most common path is to use a regular **List** as an intermediate step.

```elixir
# 1. Convert a Tuple of Tuples to a Keyword List
# A tuple containing key-value pairs can be turned into a list.
tuple = {{:name, "Budi"}, {:age, 25}}
keyword_list = Tuple.to_list(tuple)
#=> [name: "Budi", age: 25]

# 2. Convert a Keyword List to a Tuple (The Opposite Way)
# A keyword list can be converted into a tuple of tuples.
keyword_list = [name: "Budi", age: 25]
tuple = List.to_tuple(keyword_list)
#=> {{:name, "Budi"}, {:age, 25}}
```

# Exercise 1: Manipulating Maps

PRADITA University

**Goal**: Understand how to add, update, delete, and access data in a Map.

1. Create a Map with `name`: "Budi", `age`: 25, and `city`: "Jakarta".
2. Add a new key, `job`, with the value "Engineer".
3. Update the value of the `age` key to 26.
4. Remove the `city` key from the Map.
5. Print the final values of the `name` and `age` keys.

```elixir
# 1. Create the initial Map
person = %{
  "name" => "Budi", "age" => 25,
  "city" => "Jakarta"
}
# 2. Add a new key
person = Map.put(person, "job", "Engineer")
# 3. Update the value of the age key
person = Map.put(person, "age", 26)
# 4. Remove the city key
person = Map.delete(person, "city")
# 5. Access and print the values
IO.puts("Name: #{person["name"]}")
IO.puts("Age: #{person["age"]}")

# Expected Output:
# Name: Budi
# Age: 26
```

# Exercise 2: Manipulating Tuples

PRADITA University

**Goal**: Understand how to create and modify a Tuple, keeping in mind its immutable nature.

1. Create a Tuple with the data: `"Budi"`, 25, and `"Jakarta"`.
2. Access and print the second element (the age).
3. Add the new element `"Engineer"` to the Tuple.
4. Remove the second element from the Tuple and print the final result.

```
# 1. Create the initial Tuple
person = {"Budi", 25, "Jakarta"}
# 2. Access the second element
# The index is zero-based
IO.puts("Age: #{elem(person, 1)}")
# 3. Add a new element
# This returns a new, larger tuple
person = Tuple.append(person, "Engineer")
# 4. Remove the second element
# This also returns a new, smaller tuple
person = Tuple.delete_at(person, 1)
# Print the final tuple
IO.inspect(person)
# Expected Output:
# Age: 25
# {"Budi", "Jakarta", "Engineer"}
```

# Exercise 3: Manipulating Lists

PRADITA University

**Goal**: Understand how to add, delete, and access elements in a List.

1. Create a List containing the numbers 1 through 5.
2. Add the number 0 to the front of the List.
3. Remove the number 3 from the List.
4. Access and print the third element of the final List.

```elixir
# 1. Create the initial List
numbers = [1, 2, 3, 4, 5]

# 2. Add 0 to the front (prepend)
numbers = [0 | numbers]
#output => [0, 1, 2, 3, 4, 5]

# 3. Remove the number 3
numbers = List.delete(numbers, 3)
#putput => [0, 1, 2, 4, 5]

# 4. Access the third element
# Index is 0-based, so we get index 2
IO.puts("Third element: #{Enum.at(numbers, 2)}")
# Expected Output:
# Third element: 2
```

# Exercise 4: Manipulating Keyword Lists

**Goal**: Understand how to create, add, update, delete, and access elements in a Keyword List.

1. Create a Keyword List with `name`: "Budi" and `age`: 25.
2. Add a new key, `city`, with the value "Jakarta".
3. Update the value of the `age` key to 26.
4. Remove the `city` key from the Keyword List.
5. Print the final values of the `name` and `age` keys.

```elixir
# 1. Create the initial Keyword List
person_kw = [name: "Budi", age: 25]
# 2. Add a new key by prepending
person_kw = [city: "Jakarta" | person_kw]
# 3. Update the value of the age key
# This adds a new :age tuple at the front
person_kw = Keyword.put(person_kw, :age, 26)
# 4. Remove the city key
person_kw = Keyword.delete(person_kw, :city)
# 5. Access and print the values
IO.puts("Name: #{person_kw[:name]}")
IO.puts("Age: #{person_kw[:age]}")
# Expected Output:
# Name: Budi
# Age: 26
```

# Exercise 5: Using Atoms

PRADITA University

**Goal**: Understand how to use atoms in pattern matching and as keys in data structures.

1. Create an atom named `:status` with the value `:ok`.
2. Use the atoms `:ok` and `:error` for pattern matching inside a `case` statement.
3. Create a Map with atom keys, such as `:name`, `:age`, and `:city`, then access and print their values.

```elixir
# 1. Create the status atom
status = :ok
# 2. Use atoms in pattern matching
case status do
  :ok -> IO.puts("Status: Success")
  :error -> IO.puts("Status: Failure")
end
# 3. Create a Map with atom keys
# Note: %{name: "Budi"} is syntax
# sugar for %{:name => "Budi"}
person_map = %{ name: "Budi", age: 25, city:
    "Jakarta" }
# Access and print the values
IO.puts("Name: #{person_map[:name]}")
IO.puts("Age: #{person_map[:age]}")
IO.puts("City: #{person_map[:city]}")
```

# Exercise 6: The 'Person' Module

PRADITA University

**Goal**: Apply all the concepts learned in a single program.

1. Create a function `create_person/3` that accepts a Name, Age, and City.
2. The function should return a Map using atom keys.
3. Create a function `update_city/2` to change the city in the Map.
4. Create a function `delete_age/1` to remove the `:age` key.
5. Print the final result after calling the functions in sequence.

```elixir
defmodule Person do
  # Creates a Map with atom keys
  def create_person(name, age, city) do
    %{name: name, age: age, city: city}
  end

  # Updates the city value
  def update_city(person, new_city) do
    Map.put(person, :city, new_city)
  end

  # Deletes the age key
  def delete_age(person) do
    Map.delete(person, :age)
  end
end
```

# Exercise 6: Script Execution

PRADITA University

**Goal**: Apply all the concepts learned in a single program.

1. Create a function `create_person/3` that accepts a Name, Age, and City.
2. The function should return a Map using atom keys.
3. Create a function `update_city/2` to change the city in the Map.
4. Create a function `delete_age/1` to remove the `:age` key.
5. Print the final result after calling the functions in sequence.

```elixir
# --- Script Execution ---
# 1. Create the person
person = Person.create_person("Budi", 25, "Jakarta")
#=> %{age: 25, city: "Jakarta", name: "Budi"}
# 2. Update the city
person = Person.update_city(person, "Bandung")
#=> %{age: 25, city: "Bandung", name: "Budi"}
# 3. Delete the age
person = Person.delete_age(person)
# 4. Print the final result
IO.inspect(person)
#=> %{city: "Bandung", name: "Budi"}
```

# Summary

PRADITA University

- This session covered Elixir's core data structures, each with a specific purpose. Understanding when to use each one is key to writing effective Elixir code.
- **Atoms** are lightweight constants, perfect for keys and status tags like `:ok`.
- **Tuples** are for fixed-size, ordered data, offering fast element access.
- **Lists** are dynamic, ordered collections, best for sequential processing.
- **Maps** are the primary key-value store for flexible data association.
- **Keyword Lists** are ordered lists of **key–value** pairs (with atom keys) mainly used for passing options to functions.
- Remember, all these data structures are **immutable**; operations always return new data rather than changing the original.